Support Vector Machines

DS605 Lecture Notes · Lectures 25 and 26

Arpit Rana · 13 and 15 October 2026

These notes follow the lecture slides in order. All examples use ShopKart data, and every number quoted was computed with scikit-learn 1.9. Practice questions with answers are in Section 10.

1. Overview

A Support Vector Machine (SVM) separates two classes with the **widest possible street**. It can perform linear or non-linear classification, regression and outlier detection. It works best on complex but small- or medium-sized datasets: thousands of rows, not hundreds of millions.

Lecture 25 covers linear SVMs: the widest street, hard and soft margins, hinge loss and training. **Lecture 26** covers non-linear SVMs: polynomial features, the kernel trick and the dual problem.

The two ShopKart problems

Lecture	Question	Features	Data
25	Will this browsing session end in a purchase?	minutes on site x_1, product pages viewed x_2	44-session pilot (separable); 80 sessions from all cities (overlapping)
26	Will the customer buy this recommended product?	the product's two taste-map scores (its PCA scores from Lecture 20–21, measured from the centre of the customer's past purchases)	300 recommendations: 210 for training, 90 for testing

Notation

- m is the number of training instances (sessions) and n is the number of features.
- $x^{(i)}$ is the feature vector of instance i.
- $t^{(i)}$ is its label: $t = +1$ for purchase and $t = -1$ for no purchase. In scikit-learn the same labels are $y = 1$ and $y = 0$, so $t = 2y - 1$.
- w is the weight vector and b is the bias.
- $h(x) = w^\top x + b$ is the score of instance x.
- $z = t \cdot h(x)$ is the score seen from the instance's own side. It is positive when the instance is correctly classified.

2. Large margin classification

Many straight lines separate the 44 pilot sessions perfectly. A line that passes very close to some sessions is risky: a new session that lands a little to one side of an existing one would be misclassified. An SVM therefore chooses the line that stays **as far as possible from both classes**.

> The decision boundary of an SVM classifier separates the two classes and also stays as far as possible from the closest training instances. This is called **large (maximum) margin classification**.

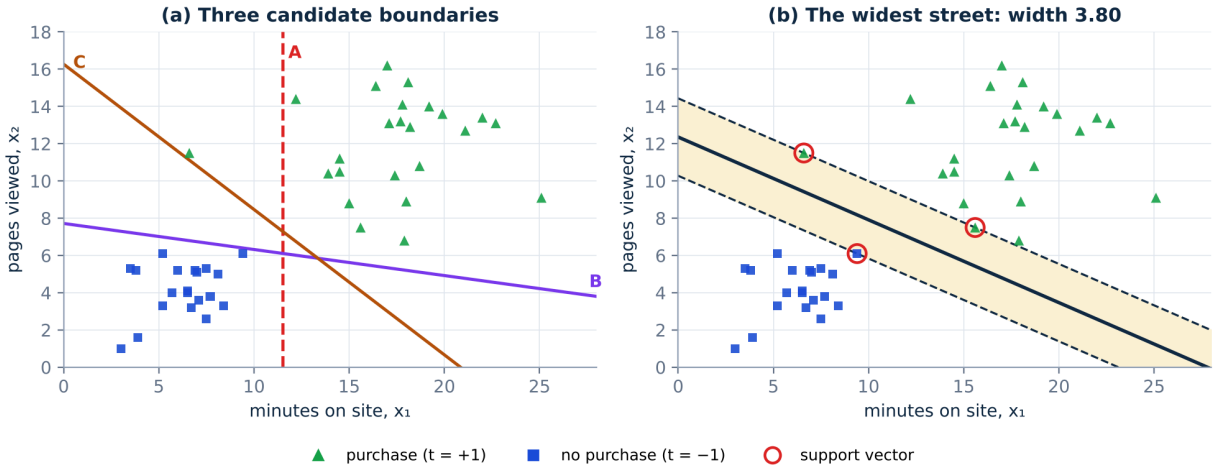


Figure 1. (a) Line A does not separate the classes; lines B and C do, but each passes very close to some sessions. (b) The SVM chooses the widest street. Three support vectors (ringed) hold it in place.

Terms

Term	Meaning
Decision boundary	The solid centre line of the street. One side predicts purchase, the other no purchase.
Margin (street)	The empty band between the two kerbs. Its width is what the SVM maximises.
Kerbs	The two dashed lines parallel to the boundary, one on each side.
Support vectors	The instances that sit on a kerb. They hold the street in place.

For the pilot, the widest street has **width 3.80** and is held by **three support vectors**: one no-purchase session at (9.4 minutes, 6.1 pages) and two purchase sessions at (15.6, 7.5) and (6.6, 11.5).

Only the support vectors matter

Adding a new session **off the street** (on its correct side) does not move the boundary at all. Deleting sessions that are not support vectors does not change the trained SVM either. The boundary is fully determined, or *supported*, by the instances on the kerbs. This also makes SVMs economical at prediction time: the model only needs to remember its support vectors.

SVMs are sensitive to feature scales

The margin is a **distance**, so it depends on the units of each feature. In Figure 2, four sessions are measured as pages viewed (1 to 5) and seconds on site (200 to 800). Seconds are hundreds of times larger than pages, so every distance is almost entirely about seconds, and the widest street ignores pages. After `StandardScaler`, both features count and a third session becomes a support vector.

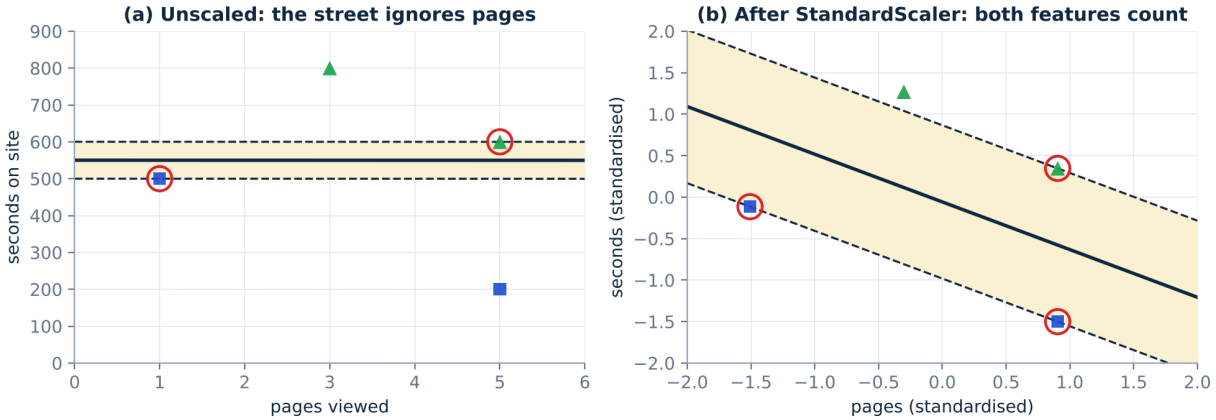


Figure 2. (a) Unscaled: the street is nearly horizontal and only two sessions are support vectors. (b) Scaled: the street uses both features.

Rule: always scale the features before training an SVM.

3. Hard and soft margins

If we insist that **every** instance is off the street and on its correct side, we have **hard margin classification**. It has two problems (Figure 3):

1. **It only works for linearly separable data.** Add one no-purchase session at 20 minutes and 15 pages to the pilot, and no straight street can separate the classes.
2. **It is very sensitive to outliers.** Add one unusual buyer at 11 minutes and 7 pages, and the street shrinks from width 3.80 to 1.77. One session out of 45 changes the policy for everyone else, and the narrow street will probably not generalise well.

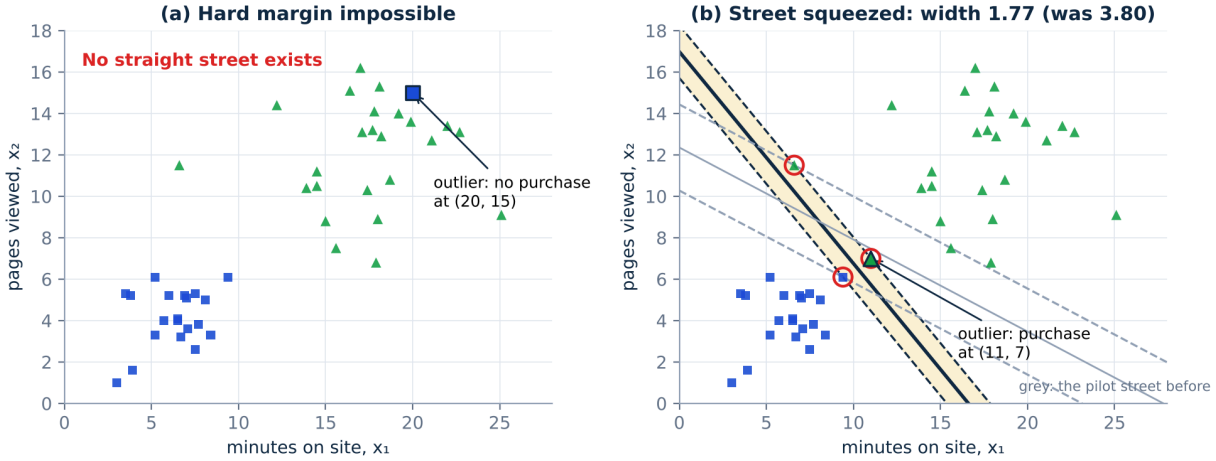


Figure 3. (a) One no-purchase outlier makes a hard margin impossible. (b) One purchase outlier squeezes the street from 3.80 to 1.77.

A more flexible model accepts a few rule-breakers in exchange for a wide, stable street. **Soft margin classification** balances two goals:

- keep the street **as wide as possible**;
- limit the **margin violations**: instances that end up inside the street or even on the wrong side.

One hyperparameter, C, controls this balance (Section 4).

Margin violation or classification error?

These are two different mistakes. Using $z = t \cdot h(x)$:

Session	Where it sits	Condition	Verdict
P	off the street, correct side	$z \geq 1$	fine
Q	inside the street, correct side	$0 \leq z < 1$	margin violation only
R	on the wrong side	$z < 0$	margin violation **and** classification error

Every classification error is a margin violation, but not every margin violation is an error. Session Q is **correctly classified**, yet it still costs something during training.

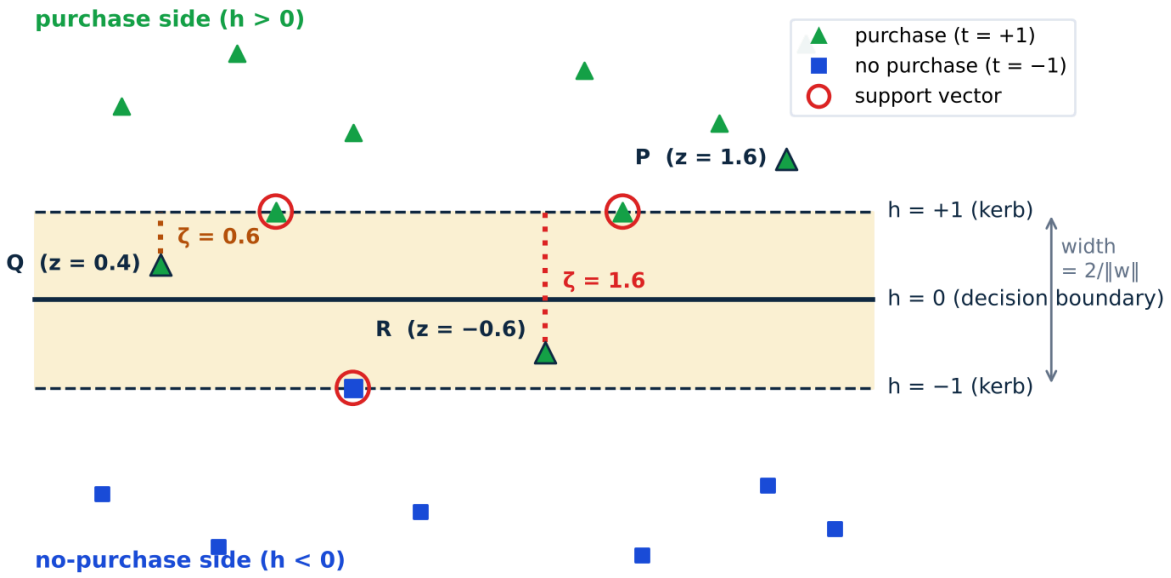


Figure 4. Anatomy of the street. The ringed sessions sit on the kerbs. P is safely off the street; Q is inside it but correctly classified; R is on the wrong side. The dotted lines show their slack ζ (Section 4).

4. The mathematics of the margin

Decision function and prediction

A linear SVM predicts the class of a new instance x by computing its decision function:

$$h(x) = w^{\top} x + b = w_1 x_1 + w_2 x_2 + \ldots + w_n x_n + b$$

It predicts **purchase** ($\hat{y} = 1$) when $h(x) \geq 0$ and **no purchase** ($\hat{y} = 0$) when $h(x) < 0$.

Worked example. The hard margin SVM on the pilot learns $w = (0.214,\ 0.481)$ and $b = -5.946$. For a session with 20 minutes and 15 pages:

$$h = 0.214 \times 20 + 0.481 \times 15 - 5.946 = 5.55 \geq 0$$

so the prediction is purchase.

The decision function is a tilted plane

Put the two features on the floor and add a vertical axis for the score. Above every point x of the floor, mark the height $h(x)$. Because h is linear in x_1 and x_2, these marks form a **flat, tilted plane** (Figure 5). The height is **signed**, like altitude measured from sea level: where h is negative, the plane lies below the floor.

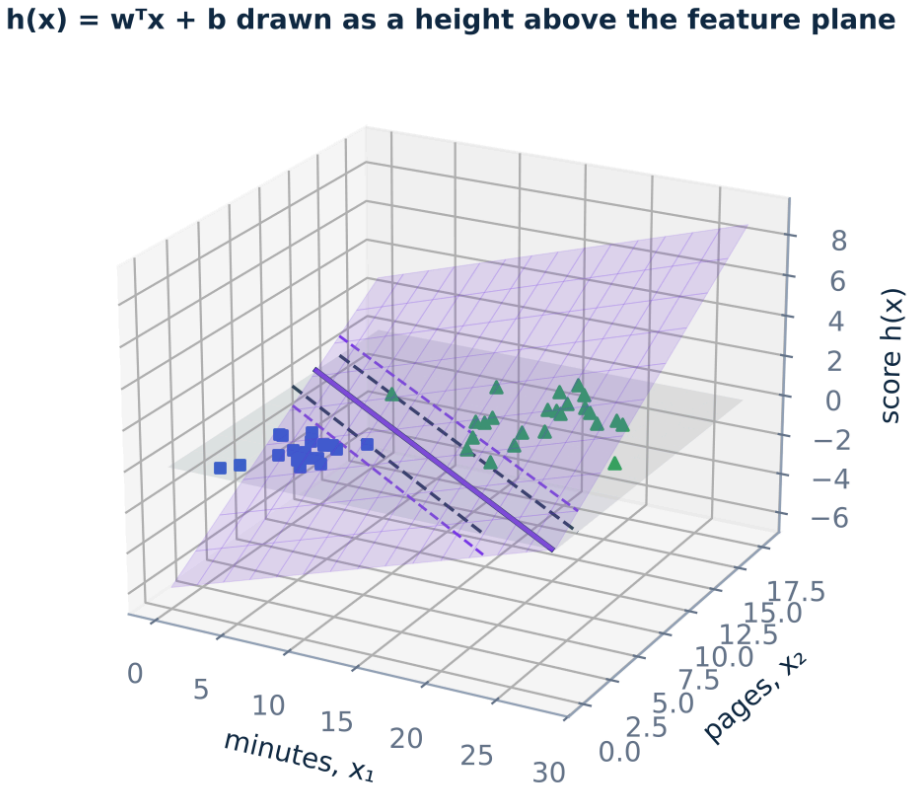


Figure 5. The pilot SVM's decision function drawn as a height above the feature plane. The boundary is where the plane crosses height 0; the kerbs are where it reaches +1 and −1.

Session (minutes, pages)	$h(x)$	Plane	Prediction
(0, 0)	−5.95	far below the floor	no purchase
(8.4, 3.3)	−2.56	below the floor	no purchase
(12, 8)	+0.47	just above the floor	purchase
(20, 15)	+5.55	well above the floor	purchase

- The **decision boundary** is where the plane crosses the floor ($h = 0$), like a shoreline.
- The **kerbs** are where the plane reaches the heights $h = +1$ and $h = -1$.

- The **sign** of h gives the class. Its **size** tells you how far from the boundary the instance is.

For a hard margin SVM, every support vector lies exactly on a kerb:

$$t^{(i)}h\left(x^{(i)}\right) = 1$$

for every support vector, and $t^{(i)}h\left(x^{(i)}\right) > 1$ for every other instance. For the pilot, $h(9.4,\ 6.1) = -1.00$, $h(15.6,\ 7.5) = +1.00$ and $h(6.6,\ 11.5) = +1.00$. The value 1 is a convention: multiplying w and b by any positive number leaves the boundary in place, so we choose the scale that puts the closest instances at height ±1.

A smaller weight gives a wider street

The plane rises at a rate of $\| w \|$ per unit moved in the direction of w. A **steeper** plane reaches ±1 sooner, so its kerbs are closer together. With one feature, $h = w_1 x_1$ reaches ±1 at $x_1 = \pm 1/w_1$: halve the slope and the kerbs move twice as far from the boundary.

How wide is the street?

Take a support vector on each kerb: x_P on the purchase kerb and x_N on the no-purchase kerb.

$$w^{\top}x_P + b = +1, \qquad w^{\top}x_N + b = -1$$

Subtracting the two equations gives $w^{\top}\left(x_P - x_N\right) = 2$. The width of the street is the part of $\left(x_P - x_N\right)$ that points along the unit normal $w/\| w \|$:

$$\left(x_P - x_N\right)^{\top}\frac{w}{\|w\|} = \frac{2}{\|w\|}$$

So **maximising the width is the same as minimising** $\| w \|$. In practice we minimise $\frac{1}{2}\| w\|^2 = \frac{1}{2}w^{\top}w$ instead. It has the same minimum, but it is differentiable everywhere ($\| w \|$ is not differentiable at $w = 0$), its gradient is simply w, and it turns the problem into a convex **quadratic programme (QP)** that standard solvers handle reliably.

For the pilot, $\| w \| = 0.527$, so the width is $2/0.527 = 3.80$. The distance on the floor from a support vector to the boundary is $|h|/\| w \| = 1/0.527 = 1.90$, half the width.

Hard margin SVM

We need $h(x) \geq 1$ for every purchase and $h(x) \leq -1$ for every non-purchase. Multiplying by $t^{(i)} = \pm 1$ turns both conditions into one:

$$\min_{w,\,b}\ \frac{1}{2}w^{\top}w$$

subject to

$$t^{(i)}\left(w^{\top}x^{(i)} + b\right) \geq 1, \qquad i = 1, 2, \ldots, m$$

In words: find the flattest plane that keeps every instance at least height 1 on its own side. There is no loss term, because the constraints forbid every violation.

Worked example: two sessions by hand. Session A = (3, 2) is no purchase, and session B = (7, 6) is a purchase.

1. By symmetry, w points from A to B: $w = c\,(1, 1)$.
2. A lies on the lower kerb: $3c + 2c + b = -1$, so $5c + b = -1$.
3. B lies on the upper kerb: $7c + 6c + b = 1$, so $13c + b = 1$.
4. Subtracting gives $8c = 2$, so $c = 0.25$ and $b = -2.25$.

Check: $\| w \| = 0.25\sqrt{2} = 0.354$, so the width is $2/0.354 = 5.66$. This equals the distance from A to B, $\sqrt{4^2 + 4^2} = \sqrt{32} = 5.66$. `SVC(kernel="linear")` returns exactly $w = (0.25,\ 0.25)$ and $b = -2.25$.

Soft margin SVM

For each instance, introduce a **slack variable** $\zeta^{(i)} \geq 0$ that measures how much instance i is allowed to violate the margin:

$$\min_{w, b, \zeta} \ \frac{1}{2} w^{\top} w + C \sum_{i=1}^{m} \zeta^{(i)}$$

subject to

$$t^{(i)}\left(w^{\top} x^{(i)} + b\right) \geq 1 - \zeta^{(i)}, \qquad \zeta^{(i)} \geq 0, \qquad i = 1, \ldots, m$$

- The first term, $\frac{1}{2} w^{\top} w$, widens the street. It is the **regularisation** term.
- The second term adds up the violations. It is the **data-fit** term.
- C is the price of one unit of slack. The two goals pull in opposite directions, and C sets the balance.

Reading ζ. At the optimum, each slack is as small as allowed:

$$\zeta^{(i)} = max\left(0,\ 1 - z^{(i)}\right)$$

ζ is measured in units of the score h, not in minutes or pages. Since the half-width of the street is $1/\ \| w \|$, ζ equals the distance of the instance from **its own kerb, divided by the half-width**. The actual distance on the floor is $\zeta/\ \| w \|$.

ζ	**Where the instance is**
0	on its own kerb or beyond it
between 0 and 1	inside the street, correct side (Q in Figure 4: $\zeta = 0.6$, 60% of the way to the boundary)
1	exactly on the decision boundary
greater than 1	on the wrong side (R in Figure 4: $\zeta = 1.6$)

C controls the trade-off

- **Small C:** violations are cheap. The street is wide, with many violations. Regularisation is **strong**.

- **Large C:** violations are expensive. The street is narrow, with fewer violations. Regularisation is **weak**.
- $C \to \infty$ gives back the hard margin.

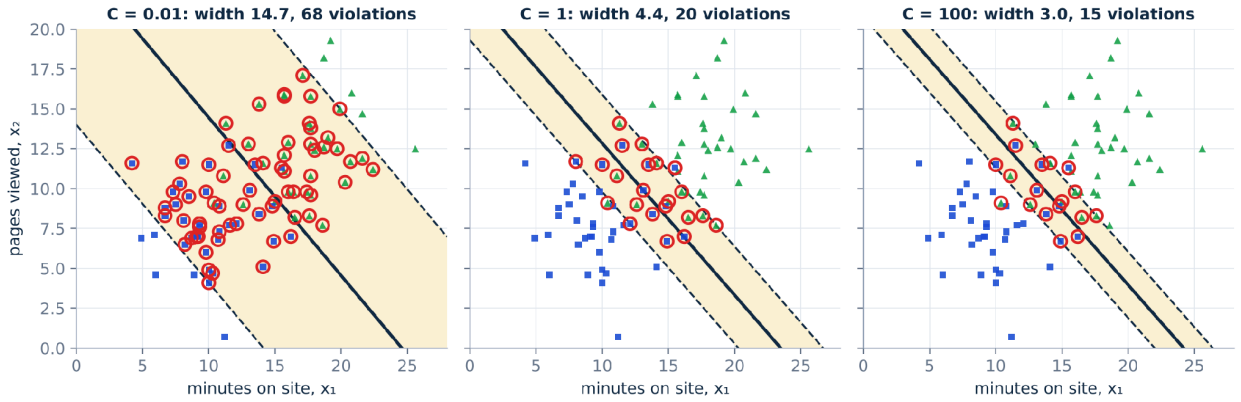


Figure 6. The soft margin SVM on 80 overlapping ShopKart sessions for three values of C. Ringed sessions are support vectors.

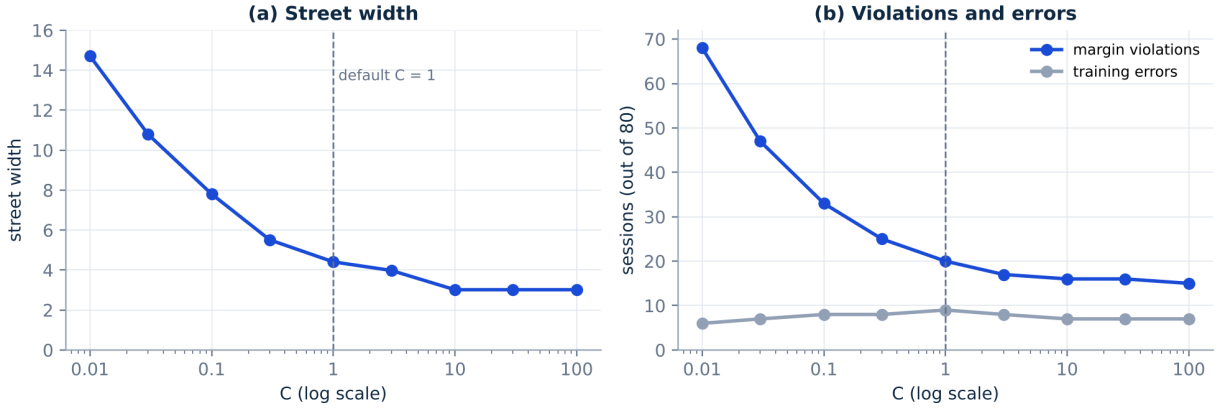


Figure 7. As C grows from 0.01 to 100, the street narrows from 14.7 to 3.0 and margin violations fall from 68 to 15, while training errors stay between 6 and 9.

Putting C on the loss is a convention. Dividing the whole objective by C gives the Ridge-style form from Lecture 16–17, with $\lambda = 1/C$:

$$\frac{1}{C} \cdot \frac{1}{2} w^{\top} w + \sum_{i=1}^{m} \zeta^{(i)}$$

Both forms have the same minimum. **If an SVM overfits, decrease C.**

5. Training in practice

Hinge loss

Substituting the optimal slack into the soft margin cost gives the **hinge loss** of one instance:

$$L(t, h(x)) = max(0,\ 1\ -\ t\, h(x))$$

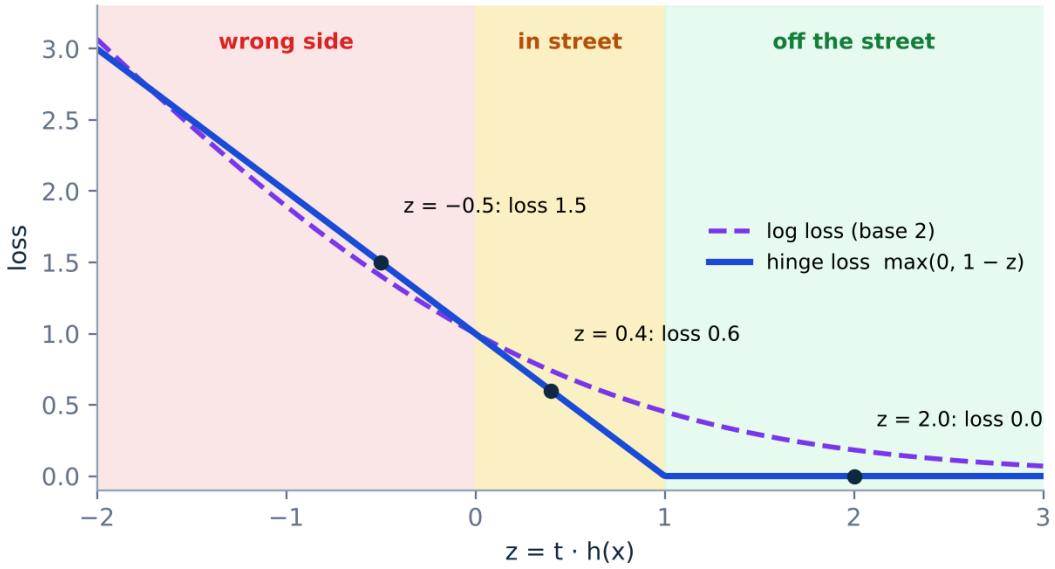


Figure 8. Hinge loss is exactly zero once an instance is safely off the street. Log loss (Lecture 11–12) never reaches zero.

z	Hinge loss	Where the instance sits	Slope
2.0	0	off the street	0
0.4	0.6	inside the street	−1
−0.5	1.5	wrong side	−1

- Instances safely off the street have zero loss and zero slope, so they do not influence training. Only instances on or inside the boundary are pulled toward the solution; these are the support vectors.
- Unlike log loss, hinge loss never rewards an instance for being even further on the correct side.
- It is not differentiable at $z = 1$. Gradient Descent can use any **subderivative** between −1 and 0 there.

The soft margin SVM is therefore the same as minimizing one unconstrained cost:

$$J(w, b) = \frac{1}{2} w^\top w + C \sum_{i=1}^{m} max\left(0,\ 1 - t^{(i)}\left(w^\top x^{(i)} + b\right)\right)$$

Linear SVM in Python

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

# X: minutes, pages (80 sessions); y: 1 = purchase, 0 = not
svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("linear_svc", LinearSVC(C=1, loss="hinge", random_state=42)),
])
svm_clf.fit(X, y)
```

```
new = [[20, 15], [12, 9], [8, 5]]
svm_clf.predict(new)            # array([1., 0., 0.])
svm_clf.decision_function(new)  # array([ 3.84, -0.64, -3.2 ])
```

- The pipeline scales the features before the SVM, for training and for prediction.
- `loss="hinge"` is the loss above. The default is `"squared_hinge"`, which is $max(0,\ 1\ -\ z)^2$: smooth at $z\ =\ 1$, and harsher on large violations. The `loss` parameter changes only the data-fit term; the $\frac{1}{2}w^{\top}w$ term (`penalty="l2"`) is always there.
- The session at 12 minutes and 9 pages has score −0.64: predicted no purchase, but **inside the street**. It is a borderline case.
- `LinearSVC` also slightly regularises the bias b, because it treats b as the weight of an extra constant feature. On scaled data this rarely matters.

An SVM score is not a probability

Unlike Logistic Regression, an SVM does not output class probabilities. Its score is a scaled, signed distance from the boundary with no fixed scale: change C or the units and every score changes, even when the predicted class does not.

Session	Score $h(x)$	Calibrated probability of purchase
25 min, 18 pages	6.39	1.00
17 min, 12 pages	1.91	0.93
14 min, 10 pages	0.32	0.57
8 min, 5 pages	−3.20	0.01

A score of 0.32 does **not** mean a 32% chance of purchase. When probabilities are needed, calibrate the scores with cross-validation (Platt scaling):

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC

prob_clf = make_pipeline(
    StandardScaler(),
    CalibratedClassifierCV(SVC(kernel="linear", C=1),
                           ensemble=False),
)
prob_clf.fit(X, y)
prob_clf.predict_proba([[14, 10]])[:, 1]   # array([0.57])
```

In scikit-learn 1.9 the older `SVC(probability=True)` is deprecated in favour of this pattern.

Three ways to train a linear SVM

Class	Time complexity	Out-of-core	Needs scaling	Kernel trick
`LinearSVC`	$O(m\ \times\ n)$	No	Yes	No
`SGDClassifier`	$O(m\ \times\ n)$	Yes	Yes	No

Class	Time complexity	Out-of-core	Needs scaling	Kernel trick
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes

- $O(m \times n)$ means that one pass computes one score per instance: n multiplications each, for m instances. Doubling the data roughly doubles the time. The total also depends on how many passes the tolerance `tol` requires.
- `SVC` compares every pair of instances, so doubling m makes it about 4 to 8 times slower. Use `LinearSVC` for a linear SVM; `SVC` is built for kernels.
- `SGDClassifier` puts its regularisation strength `alpha` on the $\frac{1}{2}w^\top w$ term and averages the loss, so the matching setting is $\alpha = 1/(mC)$.

Online SVM

An online SVM learns from one instance at a time and never stores past data. Dividing J by mC, the cost of a single instance is

$$J_i(w, b) = \frac{\alpha}{2}w^\top w + max\left(0,\ 1 - z^{(i)}\right), \qquad \alpha = \frac{1}{mC}$$

Stochastic Gradient Descent with a learning rate η gives this update for each arriving instance:

- **If** $z < 1$ (inside the street or on the wrong side): $w \leftarrow w - \eta(\alpha w - t\,x)$ and $b \leftarrow b + \eta\,t$. The street is pulled towards the instance, so it lands further on its own side next time.
- **If** $z \geq 1$ (safely off the street): $w \leftarrow w - \eta\,\alpha\,w$. Only a small shrink, which keeps $\| w \|$ small, and the street is wide.

Worked example (scaled features, starting from $w = (0, 0)$ and $b = 0$, with $\eta = 0.5$ and $\alpha = 0.01$):

1. Purchase $x = (1.2,\ 0.8)$: $h = 0$, so $z = 0 < 1$. Then $w = (0.6,\ 0.4)$ and $b = 0.5$.
2. No purchase $x = (-1,\ -0.5)$: $h = -0.3$, so $z = 0.3 < 1$. Then $w = (1.097,\ 0.648)$ and $b = 0$.
3. Purchase $x = (1.5,\ 1)$: $h = 2.29$, so $z = 2.29 \geq 1$. Only the shrink applies: $w = (1.092,\ 0.645)$.

```
from sklearn.linear_model import SGDClassifier

scaler = StandardScaler()
clf = SGDClassifier(loss="hinge", alpha=1/(m*C))
for X_batch, y_batch in stream_of_sessions():
    scaler.partial_fit(X_batch)
    clf.partial_fit(scaler.transform(X_batch), y_batch, classes=[0, 1])
```

Use it when data does not fit in memory, arrives as a stream (for example, one ShopKart session every second), or drifts over time. Its path is noisy, it needs a decreasing learning rate and scaled features, and it is linear only.

6. Non-linear data and polynomial features

On the taste map, recommended products **close to the customer's taste** are purchased, while products far **in any direction** are ignored. The bought products form a core and the ignored ones a ring around it (Figure 9). No straight street can separate them: a linear SVM scores 51% on the training products and 47% on the test products, no better than tossing a coin. This is the ring-and-core shape from Kernel PCA in Lecture 20–21.

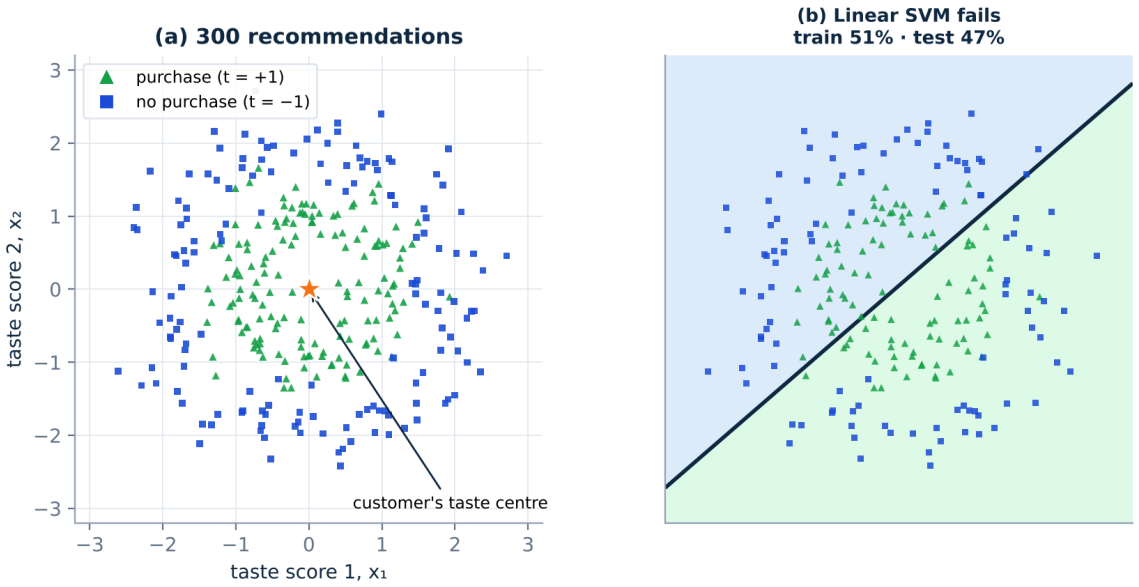


Figure 9. (a) The taste map: bought products (triangles) sit near the customer's taste centre. (b) A linear SVM cannot separate them. Shading shows the predicted class: green for purchase, blue for no purchase.

Add a feature, gain a dimension

Take only the first taste score, the **taste gap** x_1. Products at $x_1 = -2$ to 2 were bought; products at ±3 and ±4 were ignored. No single cut point separates them. Now add a second feature $x_2 = x_1^2$. Each product is lifted onto a parabola, and the classes become linearly separable (Figure 10). The SVM draws a horizontal line at $x_2 = 6.5$, halfway between $2^2 = 4$ and $3^2 = 9$. Back on the original line, the model predicts a purchase when

$$|x_1| < \sqrt{6.5} = 2.55$$

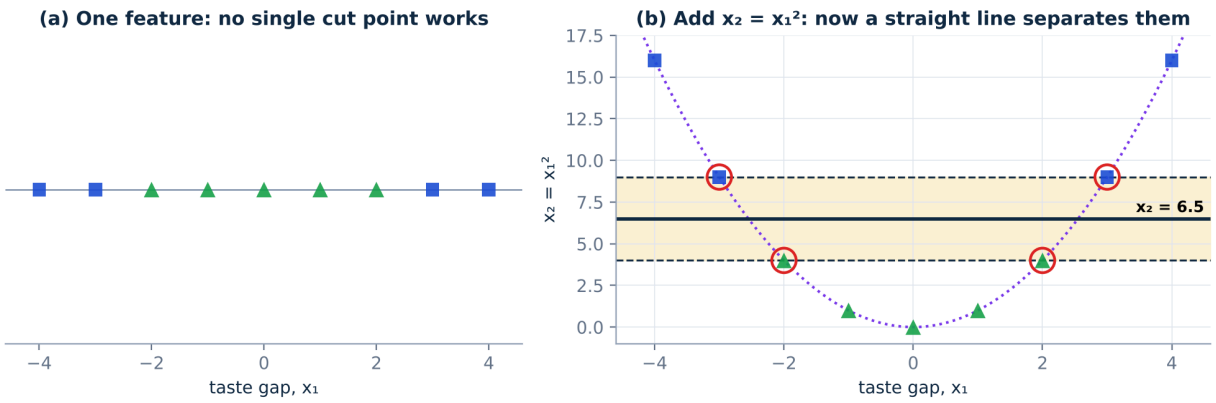


Figure 10. (a) On one feature, no single cut point separates the classes. (b) Adding $x_2 = x_1^2$ makes them separable by a straight line.

On the 2-D taste map, the rescuing feature is $x_1^2 + x_2^2$, the squared distance from the customer's taste center. We do not need to guess it: adding **all** degree-2 terms (x_1^2, $x_1 x_2$, x_2^2) lets a linear SVM find the right combination on its own.

```
from sklearn.preprocessing import PolynomialFeatures

polynomial_svm_clf = Pipeline([
    ("poly_features", PolynomialFeatures(degree=2)),
    ("scaler", StandardScaler()),
    ("svm_clf", LinearSVC(C=10, loss="hinge")),
])
polynomial_svm_clf.fit(X_train, y_train)
```

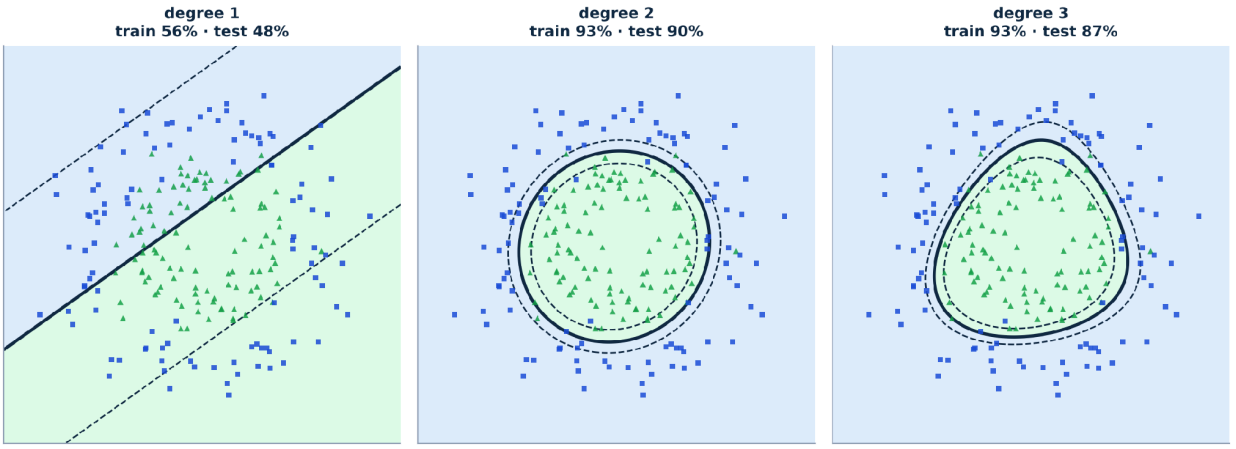


Figure 11. A linear SVM on polynomial features of degree 1, 2 and 3. Solid line: decision boundary; dashed lines: kerbs.

Adding polynomial features is simple and works with many Machine Learning algorithms, not only SVMs.

The price of polynomial features

`PolynomialFeatures(degree=d)` turns n features into

$$\frac{(n+d)!}{n!\,d!}$$

features.

For $n = 2$ and $d = 3$ this gives 10 features. For ShopKart's 100 session features at degree 5 it gives **9,65,60,646** features, nearly ten crore. A low degree cannot capture very complex patterns; a high degree creates a huge number of features, and training time grows with n: $O(m \times n)$ for `LinearSVC` and $O\left(m^2 \times n\right)$ or worse for `SVC`.

7. The kernel trick

The kernel trick yields the same result as adding many polynomial features, even of very high degree, **without ever constructing them**. Scikit-learn's `SVC` uses it.

A 2nd-degree mapping

Apply this mapping to every product on the taste map:

$$\phi(x) = \left(x_1^2,\ \sqrt{2}\, x_1 x_2,\ x_2^2\right)^{\mathsf{T}}$$

The mapped vector is three-dimensional. In this space, the bought products sit low, near the corner, and a **flat plane** separates them from the rest (Figure 12). A linear SVM in ϕ-space is a curved SVM on the taste map.

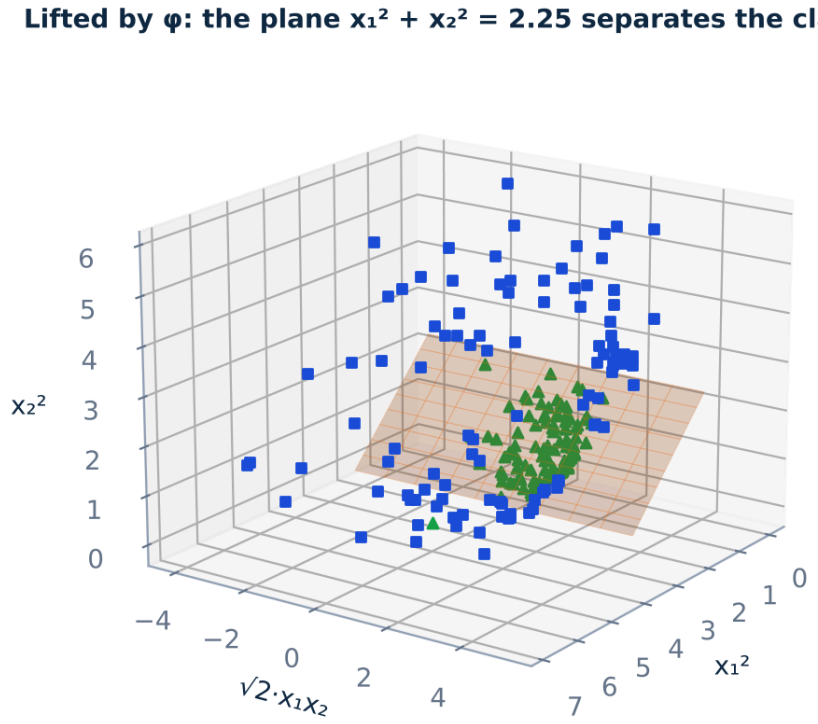


Figure 12. Each product mapped to $\phi(x)$. The plane $x_1^2 + x_2^2 = 2.25$ separates bought products (triangles) from the rest.

The dot product after the mapping

Take two products a and b, map both, and take the dot product of the mapped vectors:

$$\phi(a)^{\mathsf{T}}\phi(b) = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2$$

The right-hand side is a perfect square:

$$\phi(a)^{\top}\phi(b) = \left(a_1 b_1 + a_2 b_2\right)^2 = \left(a^{\top} b\right)^2$$

We never need to compute ϕ: we just square the ordinary dot product.

Check with numbers. Take $a = (2, 1)$ and $b = (1, 3)$.

- The long way: $\phi(a) = (4,\ 2.828,\ 1)$ and $\phi(b) = (1,\ 4.243,\ 9)$, so $\phi(a)^{\top}\phi(b) = 4 + 12 + 9 = 25$.
- The kernel way: $a^{\top} b = 2 + 3 = 5$, and $5^2 = 25$.

The two routes always agree, and the kernel route stays in the original 2-D space however large the mapped space is.

Kernel functions

A **kernel** is a function that computes the dot product of two mapped vectors using only the original vectors, without computing, or even knowing, the transformation ϕ:

$$K(a, b) = \phi(a)^{\top}\phi(b)$$

Mercer's theorem, simply: if K is symmetric and the matrix of its values $K\left(x^{(i)}, x^{(j)}\right)$ is positive semi-definite for any set of points, then some mapping ϕ exists, possibly into infinitely many dimensions. We may then use K as a measure of similarity between two instances, and the SVM behaves as if it worked in the matching ϕ-space.

Kernel	Formula	scikit-learn	Think of it as
Linear	$K(a, b) = a^{\top} b$	`kernel="linear"`	no lift at all
Polynomial	$K(a, b) = \left(\gamma\, a^{\top} b + r\right)^d$	`kernel="poly", degree=d, coef0=r`	products of features up to degree d
Gaussian RBF	$K(a, b) = e^{-\gamma\|a-b\|^2}$	`kernel="rbf", gamma=γ`	similarity that fades with distance
Sigmoid	$K(a, b) = \tanh\left(\gamma\, a^{\top} b + r\right)$	`kernel="sigmoid"`	valid only for some γ and r

The mapping above is the polynomial kernel with $\gamma = 1$, $r = 0$ and $d = 2$.

The RBF kernel measures similarity

The RBF kernel gives 1 for identical instances and falls towards 0 as they move apart. γ **sets how quickly it falls** (Figure 13).

- **Small γ:** wide bells. Each training instance influences a large region, and the boundary is smooth.

- **Large γ:** narrow bells. Each instance influences only a small region around itself, so the boundary bends around individual points and can overfit.

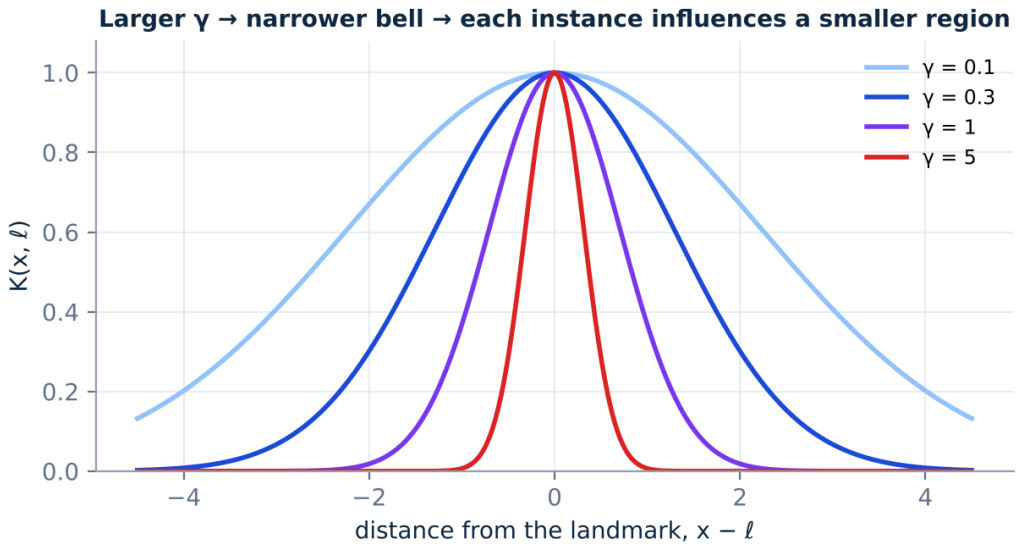


Figure 13. RBF similarity to a landmark ℓ for four values of γ.

Which kernel should you try first?

1. **Linear first.** `LinearSVC` is much faster than `SVC(kernel="linear")`, especially with many instances or many features.
2. **Then, the Gaussian RBF**, if the training set is not too large. It works well in most cases.
3. **Then the rest**, with spare time and computing power, using cross-validation and grid search.

On the taste map, the linear model reaches 47% on test products and a tuned RBF model reaches 91%.

8. The dual problem

The obstacle

With a mapping ϕ, each instance x becomes $\phi(x)$, and the boundary in the new space is the hyperplane $w^\top \phi(x) + b = 0$. The hard margin problem becomes

$$\min_{w,b} \ \frac{1}{2} w^\top w$$

subject to

$$t^{(i)}\left(w^\top \phi\left(x^{(i)}\right) + b\right) \geq 1, \qquad i = 1, \ldots, m$$

But w now lives in ϕ-space. For degree 5 on 100 features, it has almost ten crore entries; for the RBF kernel it has infinitely many. The way out is to rewrite the problem so that ϕ appears only inside dot products, which a kernel can then replace. The rewritten problem is the **dual**.

From primal to dual

Step 1: the Lagrangian. Introduce one multiplier $\alpha^{(i)} \geq 0$ per constraint:

$$L(w, b, \alpha) = \frac{1}{2} w^{\top} w - \sum_{i=1}^{m} \alpha^{(i)} \left[t^{(i)} \left(w^{\top} x^{(i)} + b \right) - 1 \right]$$

Step 2: set the derivative with respect to w to zero.

$$w = \sum_{i=1}^{m} \alpha^{(i)} t^{(i)} x^{(i)}$$

So w is a weighted sum of the training instances.

Step 3: set the derivative with respect to b to zero.

$$\sum_{i=1}^{m} \alpha^{(i)} t^{(i)} = 0$$

Step 4: substitute w back into L and flip the sign to minimise. The instances now appear only as dot products between pairs, $x^{(i)\top} x^{(j)}$. With the mapping, each of those dot products becomes a kernel:

$$\min_{\alpha} \ \frac{1}{2} \sum_{i=1}^{m} \sum_{j=1}^{m} \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} K\left(x^{(i)}, x^{(j)} \right) - \sum_{i=1}^{m} \alpha^{(i)}$$

subject to

$$\alpha^{(i)} \geq 0, \quad i = 1, \ldots, m, \qquad \sum_{i=1}^{m} \alpha^{(i)} t^{(i)} = 0$$

- In general the dual only gives a lower bound on the primal's optimal value. Here the objective is convex and the constraints are linear, so the dual has **the same solution** as the primal.
- For the soft margin, the only change is a ceiling on each multiplier: $0 \leq \alpha^{(i)} \leq C$.

Why SVMs need only dot products

The dual never asks for a single instance on its own, only for **pairs**. Put every pairwise dot product into an $m \times m$ table, the **Gram matrix** $K_{ij} = x^{(i)\top} x^{(j)}$ (Figure 14). For example, pilot sessions (8.4, 3.3) and (7.1, 3.6) give $8.4 \times 7.1 + 3.3 \times 3.6 \approx 72$.

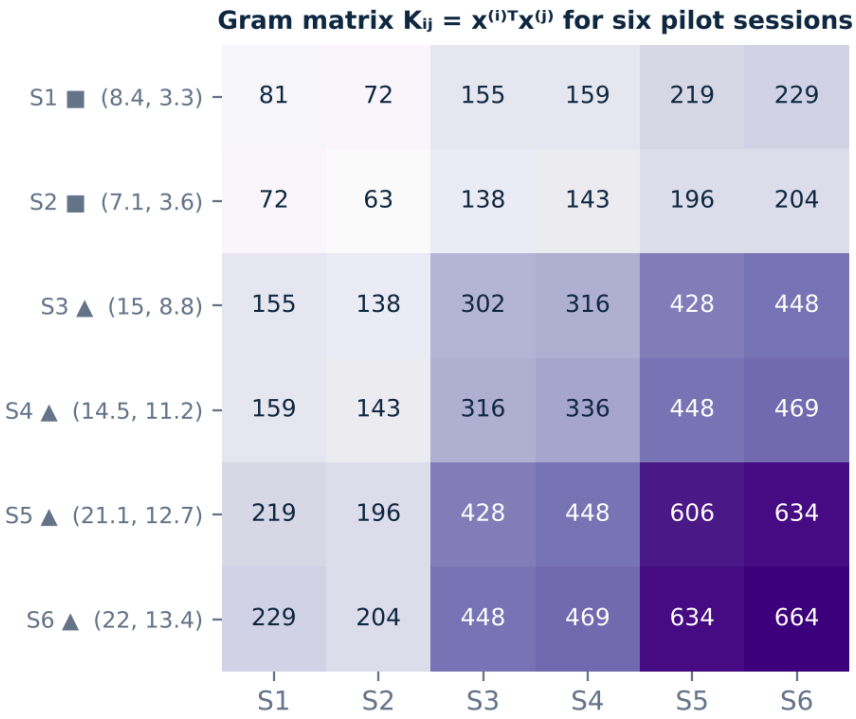


Figure 14. The Gram matrix of six pilot sessions. Training needs only this table and the labels.

- **Training** needs only this table and the labels.
- **Prediction** requires only dot products between the support vectors and the new instance (see below).
- A dot product is a **similarity**; a kernel is simply a different similarity, computed as if in a richer space. Swap every $x^{(i)\top}x^{(j)}$ for $K\left(x^{(i)}, x^{(j)}\right)$ and nothing else in the algorithm changes. This is why the kernel trick works in the dual and not in the primal.
- The table has $m \times m$ entries, which is why SVC costs between $O\left(m^2 n\right)$ and $O\left(m^3 n\right)$.

A raw dot product scales with the length of the vectors (the purchase sessions in Figure 14 have larger values only because they are longer visits), so it is a rough measure of similarity. A kernel such as an RBF, which depends only on the distance between two instances, is often preferable.

Why most α values are zero

At the optimum, every instance satisfies **complementary slackness**:

$$\alpha^{(i)}\left(z^{(i)} - 1\right) = 0$$

An instance strictly beyond its kerb has $z > 1$, so its α must be 0. Think of α as the force the instance's constraint exerts on the street: a wall the street does not touch pushes with zero force. Instances with $\alpha > 0$ are, by definition, the **support vectors**. In the pilot, 41 of the 44 sessions have $\alpha = 0$.

For the soft margin, there are three cases:

Where the instance is	z	α
beyond its kerb	$z > 1$	$\alpha = 0$

Where the instance is	z	α
on its kerb	$z = 1$	$0 \le \alpha \le C$
inside the street or on the wrong side	$z < 1$	$\alpha = C$

From α back to w and b

A QP solver returns $\hat{\alpha}$. Then, with n_s support vectors:

$$\hat{w} = \sum_{i=1}^{m} \hat{\alpha}^{(i)} t^{(i)} x^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\hat{\alpha}^{(i)} > 0} \left(t^{(i)} - \hat{w}^{\top} x^{(i)} \right)$$

Pilot support vector	t	$\hat{\alpha}$
(9.4, 6.1)	−1	0.139
(15.6, 7.5)	+1	0.067
(6.6, 11.5)	+1	0.072

Check: $\sum \hat{\alpha} t = -0.139 + 0.067 + 0.072 = 0$. For the two-session example (A = (3, 2) and B = (7, 6), both with $\alpha = 1/16$):

$$\hat{w} = \frac{1}{16}[(7,6) - (3,2)] = \frac{1}{16}(4,4) = (0.25,\ 0.25)$$

as found by hand in Section 4.

The decision rule with a kernel

Substitute $\hat{w} = \sum \hat{\alpha}^{(i)} t^{(i)} \phi\left(x^{(i)}\right)$ into $h(x) = \hat{w}^{\top} \phi(x) + \hat{b}$. The new instance appears only inside $\phi\left(x^{(i)}\right)^{\top} \phi(x)$, which the kernel computes, and every non-support vector drops out because its α is 0:

$$h(x) = \sum_{\hat{\alpha}^{(i)} > 0} \hat{\alpha}^{(i)} t^{(i)} K\left(x^{(i)}, x\right) + \hat{b}$$

Predict purchase when $h(x) \ge 0$. $\hat{b}$ is computed the same way, from kernel values between support vectors only.

Linear check (pilot, $x = (20, 15)$).

$$-0.139 \times 279.5 + 0.067 \times 424.5 + 0.072 \times 304.5 - 5.946 = 5.55$$

This is the same score as in Section 4, computed without w.

RBF example (taste gap, $\gamma = 0.3$). The SVM finds four support vectors: bought products at $x = \pm 2$ with weight $\hat{\alpha} t = +3.81$, ignored products at $x = \pm 3$ with weight -3.81, and

$\hat{b} = -0.016$. Each support vector adds a Gaussian **bump** to the score, upward for a bought product and downward for an ignored one (Figure 15).

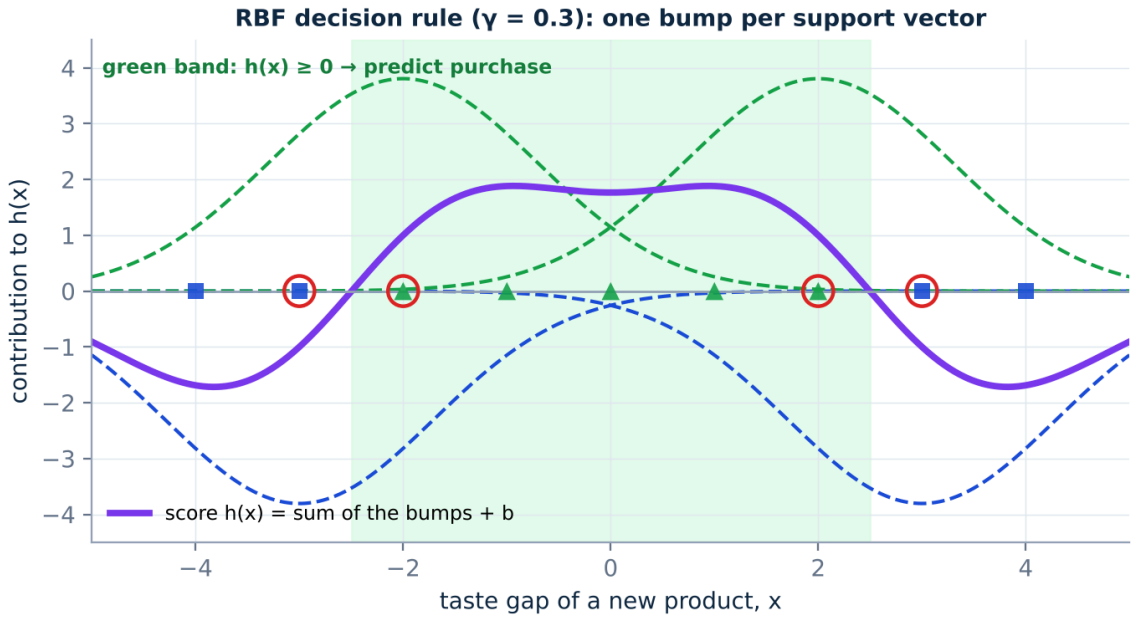


Figure 15. The RBF decision rule. Dashed curves: one bump per support vector (green up, blue down). Solid curve: their sum plus $\hat{b}$. The green band is where the model predicts purchase.

New product x	Upward bumps	Downward bumps	$h(x)$	Prediction
0	1.15 + 1.15	−0.26 − 0.26	+1.77	purchase
2	3.81 + 0.03	−2.82	+1.00	purchase, on the kerb
2.5	3.53 + 0.01	−3.53	≈ 0	the boundary
3	2.82	−3.81	−1.00	ignore, on the kerb
4	1.15	−2.82	−1.69	ignore

Prediction is a weighted vote: each support vector says “you look like me” through K, and votes for its own class with weight $\hat{\alpha}$. Far from every support vector all bumps fade, and the score drifts back to $\hat{b}$.

Primal or dual?

- **Only the dual allows the kernel trick.** The primal needs w and $\phi(x)$ explicitly.
- **The dual can be faster** when there are fewer training instances than features: it has one unknown per instance (m), while the primal has one per feature (n).
- SVC always solves the dual, which is why it supports kernels and why its cost grows with m^2 to m^3.

9. Kernels in practice

All results below use the taste map: 210 training products and 90 test products.

Polynomial kernel

```
from sklearn.svm import SVC

poly_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="poly", degree=3, coef0=1, C=5)),
])
poly_kernel_svm_clf.fit(X_train, y_train)
```

Degree d	**coef0** r	**Train**	**Test**
2	0	93%	91%
2	1	93%	90%
3	0	67%	57%
3	1	94%	87%
10	0	92%	90%
10	1	99.5%	86%

`coef0` controls how much the model is influenced by high-degree versus low-degree terms. Note the row $d = 3, r = 0$: without r, a cubic kernel has no squared terms, so it cannot draw a circle. The row $d = 10, r = 1$ fits the training products almost perfectly but drops on new ones: it overfits.

Gaussian RBF kernel

```
rbf_kernel_svm_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm_clf", SVC(kernel="rbf", gamma=5, C=1)),
])
```

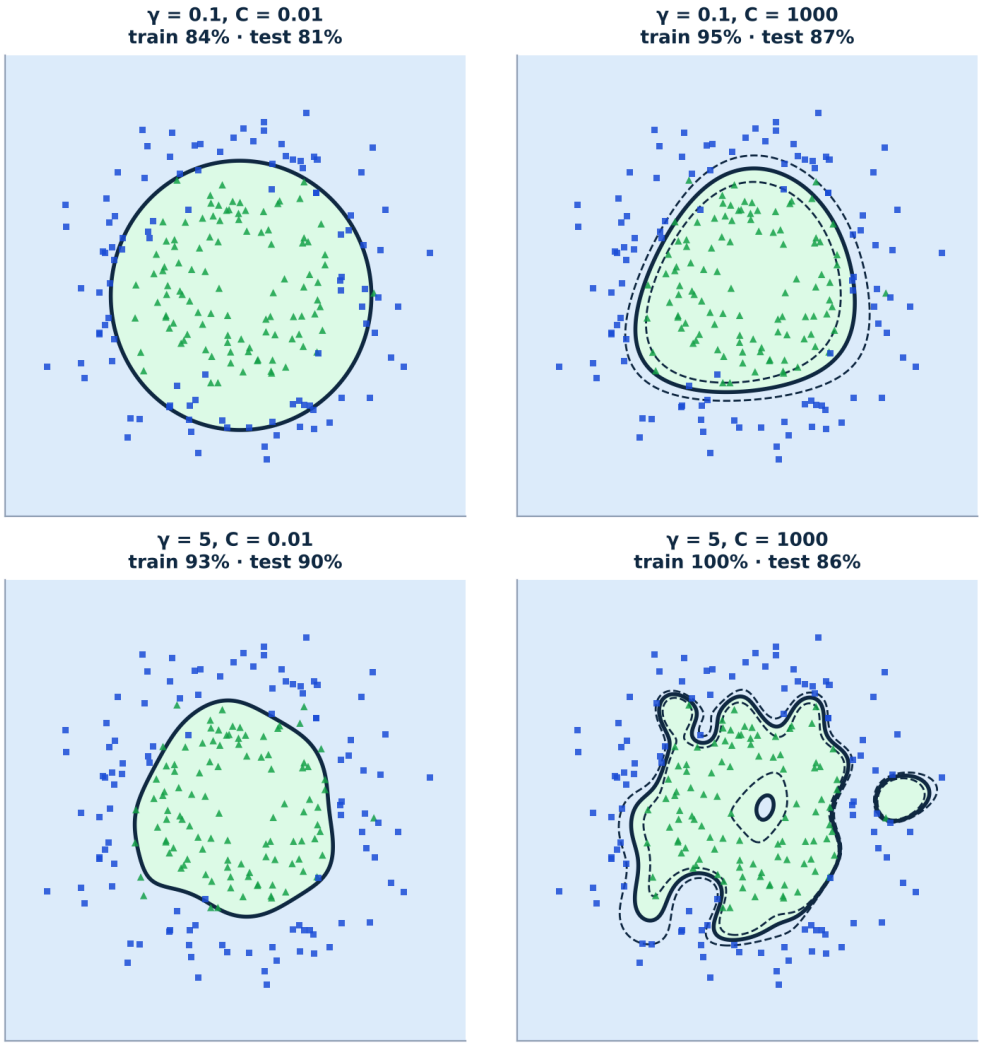


Figure 16. RBF SVMs with four settings of γ and C. Low γ and low C give a smooth boundary; high γ and high C wrap around individual products.

γ	C	Train	Test	Reading
0.1	0.01	84%	81%	too smooth: underfits
0.1	1000	95%	87%	
5	0.01	93%	90%	
5	1	96%	91%	97 support vectors
5	1000	100%	86%	wraps around individual products: overfits

Both γ and C act like regularisation dials. **Lower** either one for a smoother boundary; raise either one if the model underfits.

Tuning with grid search

```
from sklearn.model_selection import GridSearchCV

pipe = Pipeline([
    ("scaler", StandardScaler()),
```

```
    ("svm", SVC(kernel="rbf")),
])
param_grid = {
    "svm__gamma": [0.1, 0.3, 1, 3, 10],
    "svm__C": [0.1, 1, 10, 100],
}
search = GridSearchCV(pipe, param_grid, cv=5)
search.fit(X_train, y_train)

search.best_params_    # {'svm__C': 10, 'svm__gamma': 3}
search.best_score_     # 0.952
search.score(X_test, y_test)   # 0.911
```

- Search γ and C on a **log scale**: their values matter by order of magnitude.
- Grid search uses only the 210 training products. The 90 test products are scored once, at the very end.
- Result: `LinearSVC` scores 0.467 on the test products; the tuned RBF `SVC` scores 0.911.

Choosing an SVM

Situation	Use	Key hyperparameters
Classes look linearly separable, or many instances	`LinearSVC`	C
Data does not fit in memory, or arrives as a stream	`SGDClassifier(loss="hinge")`	$\alpha = 1/(mC)$
Curved boundary, up to tens of thousands of instances	`SVC(kernel="rbf")`	γ, C
Known polynomial structure	`SVC(kernel="poly")`	d, r, C
Calibrated probabilities are needed	`CalibratedClassifierCV(…)`	wrap any of the above

In every case: scale the features, and tune with cross-validation.

10. Summary and practice questions

Seven takeaways

1. An SVM chooses the **widest street** between the classes; the support vectors alone decide it.
2. The width is $2/\|w\|$. Soft margins add slack, priced by C; a smaller C means stronger regularisation.
3. Hinge loss is zero off the street. A margin violation is not always a classification error.
4. Scale the features first, and read the SVM score as a distance, not a probability.
5. Adding features such as x^2 turns curved problems into straight ones, at a price.
6. The kernel trick computes $\phi(a)^\top \phi(b)$ without building ϕ; the dual uses the data only through such dot products.
7. Try linear first, then RBF; tune γ and C by grid search.

Practice questions

Try each question before reading its answer.

1. **ShopKart deletes 30 old sessions, none of which is a support vector. Does the trained SVM change?**

   Answer: No. The street is held in place only by the support vectors.

2. **Your SVM scores 99% on training sessions but 85% on validation sessions. Should you increase or decrease C?**

   Answer: Decrease C. Violations become cheaper, so the model chooses a wider, more stable street.

3. **A session sits inside the street but on its correct side. Is it an error? Does it have a loss?**

   Answer: It is not a classification error, but it is a margin violation, so its hinge loss is between 0 and 1.

4. **Three sessions have $z = 2.0$, 0.4 and -0.5. What is the hinge loss of each, and which ones affect training?**

   Answer: 0, 0.6 and 1.5. Only the last two affect training; the first has zero loss and zero slope.

5. **ShopKart's site produces a new labelled session every second. Which class would you use, and how would you train it?**

   Answer: `SGDClassifier(loss="hinge")`, trained with `partial_fit` on small batches. `LinearSVC` and `SVC` must see all the data at once.

6. **Doubling every weight in w (and b) changes the street width how?**

   Answer: It halves the width, since the width is $2/\|w\|$. The boundary itself does not move.

7. **Why does an SVM need feature scaling, while a decision tree does not?**

Answer: The SVM maximises a distance, which depends on units. A tree compares one feature with a threshold at a time.

8. **Four sessions: purchases at (0, 1) and (1, 0), non-purchases at (0, 0) and (1, 1). Which single extra feature makes them linearly separable?**

   Answer: $x_3 = \left(x_1 - x_2\right)^2$. It is 1 for both purchases and 0 for both non-purchases. A degree-2 polynomial kernel finds this without being told.

9. **Why can the kernel trick be used in the dual but not in the primal?**

   Answer: The dual contains the data only as dot products $x^{(i)\top}x^{(j)}$, which a kernel can replace. The primal needs w and $\phi(x)$ themselves.

10. **An RBF SVM underfits. Name two changes that would help.**

    Answer: Increase γ, increase C, or both.

11. **ShopKart has 100 session features. How many features does `PolynomialFeatures(degree=5)` create?**

    Answer: $\frac{105!}{100!\,5!} = 9,65,60,646.$

12. **A session's SVM score is 0.32. Is its chance of purchase 32%?**

    Answer: No. The score has no fixed scale. Calibration gives about 0.57 for this session.